14.5.3 Πίνακας χημικών στοιχείων 327
14.5.4 Τρίλιζα 332

14.6 Ασκήσεις προς Λύση 341
14.6.1 Πολλαπλές τράπουλες 341
14.6.2 Μελέτη 341
14.6.3 Περιοδικός Πίνακας 341

Βιβλιογραφία 342

Κεφάλαιο 15 343

15 Διαχείριση Αρχείων 343

15.1. Είσοδος με την Scanner 344

15.2 Δομημένα αρχεία δεδομένων 345

15.3 Ανάγνωση και εγγραφή σε αρχεία ψηφιολέξεων 351

15.4 Αρχεία τυχαίας προσπέλασης 352

15.5 Σειριοποίηση 353

15.6 Χρήσιμες λειτουργίες 356

15.7 Λυμένες Ασκήσεις 357
15.7.1 Περιοδικός Πίνακας 357
15.7.2 Κατάλογοι αρχείων 360

15.8 Ασκήσεις προς Λύση 362
15.8.1 Quiz 2η έκδοση 362
15.8.2 Grep 362
15.8.3 CopyFile 362

Βιβλιογραφία 364

Κεφάλαιο 16 365

16 Εξαιρέσεις 365

16.1 Η στοίβα κλήσης 365

16.2 Τύποι απροσδόκητων λαθών 366

16.3 Σύλληψη εξαιρέσεων 367

16.4 Η πρόταση finally 369

16.5 Εξαιρέσεις οριζόμενες από τον χρήστη 372

16.6 Διαχείριση και σύνθετη σύλληψη 373

16.7 Βασικές προκαθορισμένες εξαιρέσεις 374

16.8 Λυμένες Ασκήσεις 376
16.8.1 Οι εξαιρέσεις ως μέρος της κανονικής ροής 376
16.8.2 Εξίσωση β’ βαθμού 377

16.9 Ασκήσεις προς λύση 378
16.9.1 Άμεση πρόσβαση με διαχείριση εξαιρέσεων 378
16.9.2 Περιοδικός Πίνακας με διαχείριση εξαιρέσεων 378
16.9.3 grep με διαχείριση εξαιρέσεων 379
16.9.4 Quiz με εξαιρέσεις οριζόμενες από τον χρήστη 379

Βιβλιογραφία 380

Κεφάλαιο 17 381

17 Γενικεύσεις 381

17.1 Ορισμός γενικευμένων κλάσεων 382

17.2 Συναρτήσεις παραμετροποιημένες ως προς τύπο 383

17.3 Παραμετροποιημένοι τύποι με όρια 384

17.4 Η κληρονομικότητα στις γενικεύσεις 385

17.5 Ο χαρακτήρας μπαλαντέρ 386

17.6 Μη Ασφαλείς Λειτουργίες 387

17.7 Λυμένες Ασκήσεις 390
17.7.1 MyHashMap<K,V> 390
17.7.2 Γενικευμένη συνάρτηση υπολογισμού αθροίσματος 395
17.7.3 Κατασκευή πινάκων 396

17.8 Ασκήσεις προς Λύση 396
17.8.1 Γενικευμένη δομή LIFO 396
17.8.2 Γενικευμένη δομή FIFO 396
17.8.3 Γενικευμένη συνάρτηση υπολογισμού γινομένου 397

Βιβλιογραφία 398

Κεφάλαιο 18 399

18 Ειδικά θέματα 399

18.1 Εκφράσεις Λάμδα 399
18.1.1 Λειτουργικές διεπαφές 399
18.1.2 Το συντακτικό των εκφράσεων λάμδα 403
18.1.3 Αναφορές μεθόδων 404

18.2 Απομνημόνευση 405

18.3 Λυμένες ασκήσεις 406
18.3.1 Εφαρμογή εκφράσεων λάμδα 407
18.3.2 Αναδρομικός υπολογισμός δύναμης με απομνημόνευση 407

Βιβλιογραφία 410

Κριτήρια Αξιολόγησης 412

Αμεταβλητότητα 412

Αρχικοποίηση 412

Η main 413

Μεταβλητές και Συναρτήσεις στιγμιότυπου 413

Αξιολόγηση βραχέως κυκλώματος 414

Μεταβλητές τιμής 415

Δισδιάστατοι πίνακες 415

Ο προκαθορισμένος Δημιουργός 416

Ψηφιολέξεις 416

Πράξεις με πραγματικούς 417

Σύλληψη εξαιρέσεων 418

instanceof 418

Υπερφόρτωση συναρτήσεων 419

Σειρά αξιολόγησης 419

Παραγωγή εξαίρεσης στο finally μπλοκ 420

Συλλογή απορριμμάτων 421

Λίστα παραμέτρων μεταβλητού μήκους 422

Auto boxing 422

Υπερφόρτωση και θεμελιώδεις τύποι 423

Αλυσιδωτή κλήση 424

Στατικές μέθοδοι 424

Μη στατικά μπλοκ αρχικοποίησης 425

Μεταβλητές αναφοράς ως παράμετροι τιμής 426

Μπλοκ αρχικοποίησης 427

Προκαθορισμένες μέθοδοι 427

Επανορισμός μεθόδων 428

Παραρτήματα 431

1. Κωδικοποίηση ακέραιων 431

2. Λέξεις κλειδιά της Java 431

Πίνακας συντομεύσεων-ακρωνυμίων

Ακρωνύμιο	Περιγραφή
JSE	Java Standard Edition
JEE	Java Enterprise Edition
JME	Java Micro Edition
JVM	Java Virtual Machine
JIT	Just In Time
JRE	Java Runtime Environment
JDK	Java Development Kit
IDE	Integrated Development Environments
API	Application Programming Interface
HTML	HyperText Markup Language
RAM	Random Access Memory
JDB	Java Data Base
RIA	Rich Internet Applications

XML	Extended Markup Language
JAF	JavaBeans Activation Framework
JPEG	Joint Photographic Experts Group
OCR	Optical Character Recognition

Πρόχειρο αντίτυπο υπο έκδοση εγχειριδίου από τις εκδόσεις Κάλλιπος

Πρόλογος

Κατά τη βιομηχανική επανάσταση, η ανθρωπότητα γνώρισε ραγδαία ανάπτυξη και πρόοδο. Εκείνη την περίοδο, ο άνθρωπος κατασκεύασε μηχανές που ήταν σε θέση να τον αντικαταστήσουν σε πλήθος χειρωνακτικών εργασιών με δραματικά αποτελέσματα στην αύξηση της παραγωγής. Στη σύγχρονη εποχή όμως, οι ρυθμοί ανάπτυξης ξεπερνούν κατά πολύ αυτούς της περιόδου εκβιομηχάνισης. Τώρα ο άνθρωπος κατασκευάζει μηχανές ικανές να υπολογίζουν, να βλέπουν, να λαμβάνουν αποφάσεις, ακόμη και να σκέπτονται ή να μιλούν.

Σήμερα, περισσότερο από κάθε άλλη εποχή της ανθρώπινης Ιστορίας, η ισχύς των νέων τεχνολογιών, θέτει μια σειρά από ερωτήματα, γεννάει ελπίδες και εγκυμονεί κινδύνους. Θα καταφέρουμε άραγε να αξιοποιήσουμε τις νέες τεχνολογίες αποκλειστικά για το καλό της ανθρωπότητας; Μήπως θα φτάσουμε σε αυτόν τον βαθμό τεχνητής ευφυΐας που η υποδούλωσή μας στις μηχανές θα καταστεί αναπόφευκτη; Μήπως οι ισχυροί του πλανήτη αξιοποιώντας την τεράστια ισχύ της τεχνολογίας καταφέρουν να ελέγξουν τους λαούς;

Τίποτα άλλο δεν μπορεί να εγγυηθεί την έκβαση της επανάστασης της Πληροφορικής παρά η πλατιά συμμετοχή όσων το δυνατόν περισσότερων πολιτών στη νέα γνώση. Όσο περισσότεροι είμαστε κοινωνοί των μυστικών της σύγχρονης τεχνολογίας τόσο ισχυρότερη είναι η άμυνά μας και τόσο μεγαλύτερη η ασφάλειά μας απέναντι σε τυχόν κακόβουλη αξιοποίησή της.

Αυτή η σκέψη αποτέλεσε το κύριο κίνητρο συγγραφής αυτού του βιβλίου. Πρόκειται για μια προσπάθεια διάχυσης της γνώσης στον νευραλγικό τομέα της Πληροφορικής, τον Προγραμματισμό. Ευελπιστούμε πως έτσι πραγματοποιούμε μια ελάχιστη συμβολή στην πρόοδο.

Αν οι ελπίδες μας έχουν κάποια βάση ή όχι, ο μόνος κατάλληλος για να το πει είστε εσείς, οι αναγνώστες αυτού του βιβλίου. Ανεξάρτητα από την ετυμηγορία σας, θέλουμε να σας ευχαριστήσουμε, γιατί χωρίς εσάς δεν θα είχε κανένα απολύτως νόημα η ύπαρξή του.

Εισαγωγή

Είναι κοινή αντίληψη πως η ανάπτυξη της πληροφορικής ακολουθεί πρωτόγνωρα ραγδαίους ρυθμούς. Μέχρι χθες δεν υπήρχαν κινητά τηλέφωνα, ενώ σήμερα είναι ολοκληρωμένοι υπολογιστές που αναγνωρίζουν το πρόσωπο του ιδιοκτήτη τους και του ανοίγουν μια πύλη εισόδου στο παγκόσμιο δίκτυο. Πέραν της ανάπτυξης της ίδιας της πληροφορικής, πληθώρα διεπιστημονικών πεδίων κάνουν την εμφάνισή τους. Η Βιοπληροφορική, η Μηχατρονική, οι Τηλεπικοινωνίες, οι Τεχνολογίες Πληροφορικής και Επικοινωνιών στην Εκπαίδευση, η Τεχνογλωσσία που συνδυάζει Γλωσσολογία και Πληροφορική, η Νανοτεχνολογία, η Νομική Πληροφορική, η Ρομποτική και άλλοι κλάδοι στους οποίους η Πληροφορική διαδραματίζει σημαίνοντα ρόλο.

Ακόμη και επιστήμες που εκ πρώτης όψεως δεν σχετίζονται με την Πληροφορική, έχουν ωφεληθεί από τις εφαρμογές της. Στους ερευνητές όλων των ειδικοτήτων, προσφέρει άμεση πρόσβαση στο σύνολο σχεδόν της ανθρώπινης γνώσης. Στην Ιατρική, προγραμματιζόμενα ρομπότ συμμετέχουν σε κρίσιμες επεμβάσεις και έμπειρα συστήματα βοηθούν σε έγκυρες διαγνώσεις. Στη Νομική, βάσεις δεδομένων εξυπηρετούν την πρόσβαση σε μεγάλες βάσεις πληροφοριών. Το ηλεκτρονικό εμπόριο επιταχύνει τις συναλλαγές αυξάνοντας το παραγόμενο προϊόν. Η Ωκεανογραφία εξοπλίζεται με νέα συστήματα που διευκολύνουν τη μελέτη των ωκεανών και των βυθών.

Ήδη με την ανάπτυξη της Τεχνητής Νοημοσύνης, τα πρώτα συστήματα που έχουν την ικανότητα να μαθαίνουν έχουν τεθεί σε λειτουργία. Ήδη ο άνθρωπος είναι κατώτερος σκακιστής σε σχέση με τον Υπολογιστή. Φαίνεται πως η Ανολοκλήρωτη Επανάσταση [1] γίνεται πραγματικότητα. Οι σύγχρονοι Υπολογιστές αναγνωρίζουν τον ιδιοκτήτη τους από το πρόσωπό του, αναγνωρίζουν τα δακτυλικά του αποτυπώματα, οι χρήστες μπορούν και επικοινωνούν με τους υπολογιστές στη δική τους ανθρώπινη γλώσσα.

Το μέλλον διαγράφεται ακόμη πιο δυναμικό. Σύμφωνα με μια δημοσίευση [2] της Kaspersky, μεγάλης εταιρείας Πληροφορικής, σε τρεις δεκαετίες, όλες οι βαριές δουλειές θα γίνονται από ρομπότ, τα σπίτια πλήρως αυτοματοποιημένα θα διεκπεραιώνουν εργασίες από την αναγκαία μαγειρική μέχρι τη διαχείριση των προμηθειών, τα αυτοκίνητα θα κινούνται αυτόνομα, οι κατασκευές επίσης θα αυτοματοποιηθούν με τη βοήθεια 3D εκτυπωτών. Ακόμη και η ίδια η ανθρώπινη φύση θα μεταβληθεί, καθώς βαδίζουμε προς τον μηχανικό άνθρωπο, δηλαδή τον άνθρωπο που θα διαθέτει τεχνητά μέλη και όργανα.

Στη βάση όλων αυτών των κατακλυσμιαίων αλλαγών βρίσκεται ο προγραμματισμός. Ο προγραμματισμός αποτελεί τη βασική οδό επικοινωνίας του ανθρώπου με τον υπολογιστή, τον τρόπο με τον οποίο ο άνθρωπος διαμορφώνει τον "νου" του άψυχου υλικού.

Η ανάπτυξη και η εξέλιξη του προγραμματισμού έχει επίσης γνωρίσει δραματικά ταχείς ρυθμούς. Πριν από μόλις τριάντα χρόνια, μία γλώσσα προγραμματισμού συμπυκνωνόταν σε ένα βιβλίο περιορισμένου όγκου και ένας επαγγελματίας του χώρου μπορούσε να καυχηθεί ότι γνωρίζει όλες τις λεπτομέρειες της γλώσσας. Σήμερα, οι περισσότερες γλώσσες προγραμματισμού έχουν μετατραπεί σε πλαίσια ανάπτυξης. Τα πλαίσια αυτά, πέραν του πυρήνα μιας γλώσσας, ενσωματώνουν βιβλιοθήκες ικανές να υποστηρίξουν πάρα πολλές λειτουργίες, από διαχείριση βάσεων δεδομένων μέχρι προγραμματισμό ενσωματωμένων συστημάτων, από τη διεθνοποίηση των προγραμμάτων μέχρι τις κανονικές εκφράσεις, από υπηρεσίες εξουσιοδότησης και αναγνώρισης μέχρι την προσαρμοσμένη δικτύωση. Έτσι οι λεπτομέρειες ενός πλαισίου, όπως της Java για παράδειγμα, είναι αδύνατο, λόγω όγκου, να συμπεριληφθούν σε ένα βιβλίο και ο καλύτερος επαγγελματίας δεν γνωρίζει με λεπτομέρεια παρά μόνο ένα ποσοστό από τις διαθέσιμες δυνατότητες.

Ο τεράστιος αυτός όγκος συνιστά μια πρόκληση και στον εκπαιδευτικό τομέα. Σε ποια επιμέρους αντικείμενα μπορεί να διαχωριστεί η αναγκαία ύλη; Με ποια σειρά θα πρέπει να δοθούν τα επιμέρους αντικείμενα στον εκπαιδευόμενο; Πώς θα πρέπει να διαμορφωθεί η ύλη σε κάθε επιμέρους αντικείμενο; Αυτά είναι κάποια από τα θεμελιώδη ερωτήματα στα οποία οφείλει να απαντήσει κάθε πρόγραμμα σπουδών εφόσον περιλαμβάνει προγραμματισμό. Για να δοθεί μια εικόνα στον αναγνώστη αναφέρουμε ως παράδειγμα ότι στο Τμήμα Πληροφορικής του ΔΙΠΑΕ υπάρχουν δέκα (10) μαθήματα εκμάθησης αποκλειστικά του Προγραμματισμού. Πιο συγκεκριμένα, Εισαγωγή στον προγραμματισμό με C++, Εισαγωγή στην Java, Αντικειμενοστρεφής Προγραμματισμός, Προγραμματισμός Διεπαφής Χρήστη, Προηγμένα Θέματα Προγραμματισμού, Προγραμματισμός του Παγκόσμιου Ιστού, Ανάπτυξη Προηγμένων Εφαρμογών Κινητών Συσκευών, Λογική και Λογικός Προγραμματισμός, Προγραμματισμός Δικτύων και τέλος Σχεδιαστικά Πρότυπα. Πέραν αυτών ένα μεγάλο πλήθος μαθημάτων, όπως η Τεχνητή Νοημοσύνη, οι Βάσεις Δεδομένων, η Τεχνητή Όραση, τα Αυτοκινούμενα Ρομπότ, τα Νοήμονα Ρομπότ, τα Λειτουργικά Συστήματα, οι Αλγόριθμοι Βελτιστοποίησης, οι Αλγόριθμοι Βιοπληροφορικής κ.ά συμπεριλαμβάνουν τον προγραμματισμό, όχι απλώς ως

εφαρμογή της διδακτέας ύλης των μαθημάτων που αφορούν αποκλειστικά τον Προγραμματισμό, αλλά διδάσκοντας νέα προγραμματιστικά εργαλεία απαραίτητα για κάθε ένα από αυτά τα αντικείμενα.

Μέσα σε αυτήν την πληθώρα θεμάτων, το εγχειρίδιο αυτό έχει ως στόχο να καλύψει το Διαδικαστικό και το Αντικειμενοστρεφές μοντέλο με την Java. Σε αυτό το επίπεδο, κρίσιμο είναι το ερώτημα της διάρθρωσης της ύλης. Η αλήθεια είναι ότι κυκλοφορούν αρκετά βιβλία που καλύπτουν τη συγκεκριμένη θεματική περιοχή. Στη μεγάλη πλειοψηφία τους παρουσιάζουν την ύλη παράλληλα, εμπλέκοντας έννοιες του διαδικαστικού με του αντικειμενοστρεφούς μοντέλου. Δεν μπορεί κανείς να θεωρήσει ότι πρόκειται για λανθασμένη επιλογή. Πολλοί συγγραφείς θεωρούν ότι κάποιες έννοιες πρέπει να διδαχθούν από την αρχή, καθώς έτσι διευκολύνεται η έγκαιρη εμπέδωσή τους. Ωστόσο, η δική μας προσέγγιση διαφέρει. Ιστορικά, πρώτα αναπτύχθηκε το διαδικαστικό μοντέλο και στη συνέχεια το αντικειμενοστρεφές. Αυτό δεν σημαίνει όμως πως το διαδικαστικό μοντέλο είναι παρωχημένο. Αντίθετα, εμπεριέχεται και αποτελεί τη βάση και του αντικειμενοστρεφούς αλλά και σχεδόν όλων των αντικειμένων προγραμματισμού που αναφέρθηκαν προηγουμένως. Επιπλέον, το διαδικαστικό μοντέλο είναι αυτοδύναμο, μπορεί να διδαχθεί αυτόνομα και σε γενικές γραμμές, οι έννοιές του είναι απλούστερες και άρα πιο προσιτές στον εκπαιδευόμενο προγραμματιστή. Βέβαια, η Java είναι μια καθαρά αντικειμενοστρεφής γλώσσα. Αυτό όμως δεν σημαίνει πως δεν συμπεριλαμβάνει το διαδικαστικό μοντέλο. Αξιολογώντας αυτά τα δεδομένα, αποφασίσαμε να οργανώσουμε την ύλη κατά τέτοιο τρόπο ώστε να εξαντλήσουμε πρώτα τη μελέτη όλων των διαδικαστικών δομών και εννοιών και στη συνέχεια να προχωρήσουμε στο αντικειμενοστρεφές μοντέλο. Ευελπιστούμε πως αυτή η διάρθρωση διευκολύνει την πρόσληψη της ύλης και την κατανόηση των εννοιών. Σε αυτό το πλαίσιο προσπαθήσαμε να δώσουμε στον αναγνώστη όλες τις αναγκαίες πληροφορίες με τη σωστή σειρά, αποφεύγοντας τη φλυαρία που κατά την άποψή μας κάνει τη μελέτη κουραστική.

Πέρα από αυτήν τη βασική επιλογή, θα θέλαμε να επιστήσουμε την προσοχή του αναγνώστη στις ακόλουθες κατευθυντήριες γραμμές μελέτης:

Ορολογία. Δίνουμε μεγάλη σημασία στη γνώση της ορολογίας του προγραμματισμού. Με λίγο φιλοσοφική διάθεση μπορούμε να πούμε ότι γλώσσα και σκέψη δεν είναι παρά οι δύο όψεις του ίδιου νομίσματος. Άραγε μπορούμε να επεξεργαστούμε σύνθετες σκέψεις χωρίς να τις οργανώσουμε λεκτικά; Μπορούμε να σκεφτούμε κάτι έστω και απλό χωρίς να το αναπαραστήσουμε νοητικά με λέξεις; Πώς θα κατανοήσουμε μια έννοια στον Προγραμματισμό που περιέχει στον ορισμό της άλλες απλούστερες έννοιες αν δεν τις έχουμε ήδη κάνει κτήμα μας;

Πράξη. Η πράξη είναι εξαιρετικά σημαντική στην εκμάθηση του Προγραμματισμού. Όσα βιβλία και να διαβάσω, όση αποφασιστικότητα και να διαθέτω για να εντρυφήσω στις έννοιες του προγραμματισμού, ελάχιστα θα καταφέρω αν δεν προγραμματίζω. Προγραμματίζοντας έχω την ευκαιρία να βλέπω τα λάθη και τις παραλείψεις μου, να αναπτύσσω την απαραίτητη αλγοριθμική σκέψη και να εξοικειώνομαι με τις αναγκαίες έννοιες στην πράξη. Όμως η πράξη δεν θα πρέπει να είναι τυχαία και άναρχη, αντίθετα θα πρέπει να διακρίνεται από συστηματικό χαρακτήρα. Θα δώσω ένα παράδειγμα για να εξηγήσω σαφώς τι εννοώ. Συχνά βρίσκομαι μάρτυρας του εξής περιστατικού: Το πρόγραμμα κάποιου εκπαιδευόμενου παρουσιάζει κάποιο πρόβλημα. Ο εκπαιδευόμενος αρχίζει να δοκιμάζει τυχαίες αλλαγές στον κώδικα με σκοπό να εξαλείψει το πρόβλημα. Κάποια στιγμή, το πρόβλημα δεν εμφανίζεται πλέον και ο εκπαιδευόμενος θεωρεί ότι το πρόβλημα λύθηκε. Πρόκειται για λανθασμένη εφαρμογή της μεθόδου ‘προσπάθησε και κάνε λάθος’ (try and error). Με αυτόν τον τρόπο είναι πιθανό πως το πρόβλημα δεν λύθηκε καν αλλά απλώς δεν εμφανίζεται κάτω από συγκεκριμένες προϋποθέσεις. Αν αυτές αλλάξουν, το πρόβλημα ενδέχεται να εμφανιστεί εκ νέου. Ακόμη όμως και αν το πρόβλημα λυθεί, ο εκπαιδευόμενος δεν κατανόησε τι ακριβώς δημιούργησε το πρόβλημα. Σε πρώτη ευκαιρία θα το επαναλάβει σε κάποιο άλλο σημείο του κώδικα. Αντί αυτής της πρακτικής, μελετήστε με προσοχή τα μηνύματα που αναφέρονται στο πρόβλημα και ψάξτε συστηματικά στον κώδικά σας μέχρι να καταλάβετε ποιο ακριβώς είναι το θέμα. Μόνο τότε προχωρήστε σε επέμβαση στον κώδικα, όταν είστε σίγουροι πως έχετε καταλάβει σωστά. Με αυτόν τον τρόπο, και θα παρέχετε πιο αξιόπιστη λύση, και θα ενσωματώσετε μια γνώση που θα αποδειχτεί σίγουρα χρήσιμη.

Θεωρία. Η πράξη είναι απαραίτητη. Το ίδιο απαραίτητη είναι και η θεωρία. Πολλοί αρχάριοι προγραμματιστές δεν δίνουν την πρέπουσα σημασία στη θεωρία. Η αλήθεια είναι πως στον προγραμματισμό, γνωρίζοντας μερικά βασικά στοιχεία μπορεί να επιχειρήσει κανείς να κάνει αρκετά πράγματα. Αυτό το χαρακτηριστικό, συχνά δίνει την εντύπωση πως δεν απαιτούνται και ιδιαίτερες γνώσεις για να προγραμματίσουμε. Πρόκειται όμως για εντελώς λανθασμένη αντίληψη. Δεν αρκεί να μπορώ να κάνω κάτι αν δεν γνωρίζω έναν σχετικά οικονομικό τρόπο για να το πετύχω. Δεν χρησιμεύει σε τίποτα να μπορώ να πάω από την Αθήνα στη Θεσσαλονίκη αν ο μόνος δρόμος που γνωρίζω περνάει από τον Βόρειο Πόλο. Η βαθιά γνώση

της θεωρίας είναι αυτή που θα διαφοροποιήσει τον αυριανό επαγγελματία προγραμματιστή από τον αιώνιο ερασιτέχνη.

Αγγλικά. Ένα άλλο σημείο στο οποίο θα ήθελα να επιστήσω την προσοχή στον επίδοξο εκπαιδευόμενο προγραμματιστή είναι η αναγκαιότητα γνώσης των Αγγλικών. Τυπικά, οι νέες εξελίξεις στον χώρο ανακοινώνονται στα Αγγλικά. Στη συντριπτική πλειοψηφία τους χώροι του διαδικτύου στους οποίους μπορείτε να λύσετε τις απορίες σας και να ενημερωθείτε σχετικά για ποικίλα θέματα Προγραμματισμού, χρησιμοποιούν τα Αγγλικά. Γνωρίζω ότι πολλοί σπουδαστές αντιμετωπίζουν δυσκολίες με τα Αγγλικά. Δυστυχώς όμως, στον τομέα του Προγραμματισμού, η γνώση των Αγγλικών είναι αναγκαία. Όμως αυτό δεν θα πρέπει να αποθαρρύνει κανέναν. Δεν απαιτείται να γνωρίζουμε Αγγλικά στο επίπεδο του να διαβάζουμε ή να γράφουμε περίπλοκα λογοτεχνικά ή φιλοσοφικά δοκίμια. Η ορολογία του χώρου είναι σχετικά περιορισμένη. Αν επιμείνετε να διαβάζετε προγραμματισμό στα Αγγλικά, σύντομα θα νοιώθετε άνετα και θα κατανοείτε τις σχετικές έννοιες. Εξάλλου, σήμερα, χάρη στην ανάπτυξη της Πληροφορικής, έχετε στη διάθεσή σας διάφορα γλωσσικά εργαλεία που μπορούν να σας βοηθήσουν σε αυτήν την προσπάθεια, π.χ. ο μεταφραστής της Google (Google Translator).

Στη συνέχεια, αυτό το εγχειρίδιο οργανώνεται ως ακολούθως: Τα κεφάλαια 1 έως και 9 μας εισάγουν στον διαδικαστικό προγραμματισμό με Java. Τα υπόλοιπα κεφάλαια αφορούν το αντικειμενοστρεφές μοντέλο. Αμέσως μετά το κεφάλαιο 9, παρατίθεται μια σειρά από κριτήρια αξιολόγησης. Παρόμοια κριτήρια αξιολόγησης δίνονται και στο τέλος του βιβλίου. Τα κριτήρια αξιολόγησης έχουν αναπτυχθεί ώστε να στοχεύουν αφενός να πληροφορήσουν τον αναγνώστη κατά πόσο έχει κάνει κτήμα του την αντίστοιχη ύλη, αφετέρου να του επισημάνουν πως στον Προγραμματισμό υπάρχουν λεπτομέρειες που είναι αναγκαίο να γνωρίζουμε.

Με αυτές τις σκέψεις, κλείνω την εισαγωγή και σας καλωσορίζω στον δημιουργικό κόσμο του Προγραμματισμού!

Κεφάλαιο 1

Σύνοψη
Παρουσιάζονται καταρχάς ιστορικά στοιχεία σχετικά με τη δημιουργία, την εξέλιξη, τους στόχους και τις βασικές σχεδιαστικές επιλογές της Java. Στη συνέχεια, εξηγούνται οι βασικές έννοιες: Γλώσσα μηχανής (Machine language), Assembly, Μεταγλωττιστής (Compiler), Διερμηνευτής (Interpreter), Διασυνδέτης (Linker). Παρουσιάζεται η αρχιτεκτονική της Java και εξηγείται η λειτουργία των συστατικών της, της Εικονικής Μηχανής (Java Virtual Machine), του Περιβάλλοντος Εκτέλεσης (Java Runtime Environment), της Εργαλειοθήκης Ανάπτυξης (Java Development Kit), του μεταγλωττιστή Just In Time. Επισημαίνεται ο διαπλατφορμικός χαρακτήρας της Java. Επιπλέον, παρουσιάζονται οι βασικές αρχές σχεδίασης και οι τεχνολογίες που υποστηρίζονται στο βασικό πλαίσιο της Java. Πιο συγκεκριμένα, αναφέρονται το πλαίσιο σύνδεσης με βάσεις δεδομένων, η ενσωματωμένη βάση δεδομένων, το πλαίσιο ανάπτυξης για ενσωματωμένα συστήματα (Java SE Embedded), τα πλαίσια ασφάλειας (Security), τα πλαίσια διεθνοποίησης (Internationalization), ανάπτυξης εφαρμογών δικτύου (Custom Networking), διανομής (Deployment), κανονικών εκφράσεων (Regular Expressions), τα πλαίσια διαδικτυακών εφαρμογών (Java Servlet API, JavaServer Pages Technology, JavaServer Pages Standard Tag Library, JavaServer Faces Technology, Java Message Service API, JavaMail API and the JavaBeans Activation Framework), το πλαίσιο διαχείρισης XML αρχείων (Java API for XML Processing), τα πλαίσια ανάπτυξης εφαρμογών με παραθυρική διεπαφή (Swing και JavaFx) και εμπλουτισμένων εφαρμογών διαδικτύου (Rich Internet Applications), η δυνατότητα για ανάπτυξη εφαρμογών κινητών και διαπλατφορμικών εφαρμογών.

Προαπαιτούμενη γνώση
Δεν απαιτούνται ειδικές γνώσεις για αυτήν την ενότητα.

Λέξεις κλειδιά
Γλώσσα μηχανής, Μεταγλωττιστής, Εικονική Μηχανή

1 Ιστορικά στοιχεία και βασικές έννοιες

Δύο είναι τα βασικά συστατικά μέρη ενός υπολογιστικού συστήματος, το υλικό (hardware) και το λογισμικό (software). Το υλικό, όπως φανερώνει και το όνομά του περιλαμβάνει εκείνο το τμήμα που έχει υλική υπόσταση. Με άλλα λόγια, το υλικό είναι μια μηχανή. Ωστόσο, πρόκειται για μια ιδιαίτερη μηχανή, θεμελιωδώς διαφορετική από όλες τις μη υπολογιστικές μηχανές. Όλες οι άλλες μηχανές σχεδιάζονται για να εκτελούν μια προκαθορισμένη εργασία. Αντίθετα, το υλικό του υπολογιστή από μόνο του δεν εκτελεί καμία εργασία. Μπορεί όμως να εκτελέσει οποιαδήποτε εργασία του υποδειχθεί από το λογισμικό. Τι είναι όμως το λογισμικό; Ένας απλός ορισμός λέει πως το λογισμικό ενός υπολογιστή είναι το σύνολο των προγραμμάτων που τρέχουν σε αυτόν. Τι είναι όμως ένα πρόγραμμα; Ένα πρόγραμμα στην απλούστερη μορφή του, δηλαδή εκφρασμένο στη γλώσσα της μηχανής, είναι ένα σύνολο εντολών τέτοιων ώστε κάθε εντολή να είναι εκτελέσιμη από το υλικό. Πρόκειται για εντολές που αντιστοιχούν σε στοιχειώδεις λειτουργίες. Ωστόσο, οι εντολές αυτές μπορούν να οργανωθούν σε αναρίθμητους συνδυασμούς έτσι ώστε κάθε συνδυασμός να παράγει ένα εντελώς διαφορετικό αποτέλεσμα. Τελικά, η μηχανή καταφέρνει και εκτελεί πολλές και διαφορετικές μεταξύ τους εργασίες.

Έτσι λοιπόν με τη βοήθεια του λογισμικού, το υλικό αποκτά χαρακτηριστικά που αντίστοιχά τους συναντά κανείς μόνο στην ανθρώπινη σκέψη, όπως για παράδειγμα η λήψη αποφάσεων ανάλογα με τα διαθέσιμα δεδομένα. Από αυτό το σημείο ξεκινάει η μεγάλη περιπέτεια της σύγχρονης πληροφορικής η οποία από τη γέννησή της μέχρι σήμερα έχει γνωρίσει μια άνευ προηγουμένου ανάπτυξη, ενώ ταυτόχρονα συμβάλλει τα μέγιστα στην ταχύτατη εξέλιξη και των υπόλοιπων επιστημών.

Όπως όμως σε όλα τα μεγάλα επιτεύγματα της ανθρώπινης επινόησης, έτσι και στον τομέα της ανάπτυξης λογισμικού δεν έλειψαν τα προβλήματα. Σύντομα, μετά τη συγγραφή των πρώτων προγραμμάτων, έγινε αντιληπτή η ανάγκη να αναπτυχθούν γλώσσες με τις οποίες να μπορούν οι άνθρωποι να προγραμματίζουν εύκολα τους υπολογιστές. Μια από αυτές τις γλώσσες είναι και η Java.

Στη συνέχεια αυτού του κεφαλαίου, αφού δώσουμε συνοπτικά τα κύρια ιστορικά στοιχεία για τη γέννηση και εξέλιξη της Java, προχωρούμε στην παρουσίαση βασικών εννοιών αναγκαίων για την κατανόηση του προγραμματισμού.

1.1 Ιστορικά στοιχεία

Το 1991, ο James Gosling στην Sun Microsystems, ανέλαβε τον σχεδιασμό μιας νέας γλώσσας, προδρόμου της Java, με το όνομα Oak [3]. Στόχος του ήταν να αξιοποιηθεί η Oak για τον προγραμματισμό οικιακών συσκευών. Προέκυψε λοιπόν η ανάγκη ένα λογισμικό που θα γραφόταν για συγκεκριμένη συσκευή, π.χ. τηλεόραση ενός κατασκευαστή, να τρέχει και σε οποιοδήποτε άλλο μοντέλο του ίδιου ή άλλου κατασκευαστή. Από την ανάγκη αυτή πηγάζει ο διαπλατφορμικός χαρακτήρας της Java. Ήδη το 1994, η Oak είχε μετονομαστεί σε Java και άρχισε να διαμορφώνεται κατάλληλα ώστε να υποστηρίζει διαδικτυακές εφαρμογές. Σύντομα, οι κατασκευαστές λογισμικού πλοήγησης στο διαδίκτυο με πρώτη την Netscape Communications άρχισαν να υποστηρίζουν τη νέα γλώσσα [4].

Το 1996, η Sun Microsystems παρουσίασε την πρώτη δημόσια έκδοση της γλώσσας. Στο τέλος του 1998, η ίδια εταιρεία, παρουσίασε την Java 2 που περιλάμβανε τρεις διαφορετικές διαμορφώσεις, τη βασική, τη διαμόρφωση για εταιρείες και τη διαμόρφωση για προγραμματισμό συσκευών συμπεριλαμβανομένων των κινητών τηλεφώνων. Οι εκδόσεις αυτές εξελίχθηκαν και ονομάστηκαν το 2006 από την Sun ως Java Standard Edition (JSE), Java Enterprise Edition (JEE) και Java Micro Edition (JME), αντίστοιχα [5]. Το 2007 η Java υιοθετείται από την Google ως βασική γλώσσα για ανάπτυξη εφαρμογών Android [6]. Την ίδια εποχή περίπου, ο Chris Oliver στην εταιρεία SeeBeyond υλοποιεί ένα πλαίσιο ανάπτυξης παραθυρικών διεπαφών που ονόμασε F3. Το 2008, η Sun αγοράζει την SeeBeyond και προσθέτει στην οικογένεια της Java την εξέλιξη της F3 που ονόμασε JavaFx [7].

Από τη γέννησή της έως σήμερα η Java βελτιώνεται και επαυξάνεται διαρκώς. Η JSE κατά τον χρόνο συγγραφής αυτού του κειμένου βρίσκεται στην έκδοση 17. Οι κώδικες που παρουσιάζονται σε αυτό το βιβλίο μεταγλωττίζονται με επιτυχία από την έκδοση 8 ή μεταγενέστερη.

1.2 Βασικές Έννοιες

Γλώσσα Μηχανής: Η θεμελιώδης γλώσσα προγραμματισμού είναι η λεγόμενη Γλώσσα Μηχανής (Machine Language). Το “αλφάβητο” αυτής της γλώσσας είναι οι χαρακτήρες 0 και 1, δηλαδή τα δυαδικά ψηφία (binary digits ή bits). Με συνδυασμό των bits σχηματίζεται το λεξιλόγιο της μηχανής, γνωστό ως σύνολο εντολών (Instruction Set). Επομένως, τα προγράμματα που είναι γραμμένα σε γλώσσα μηχανής είναι ακολουθίες από bits,γεγονός που καθιστά τη συγγραφή αλλά και την ανάγνωσή τους ιδιαίτερα δύσκολη. Επιπλέον, το σύνολο των εντολών διαφέρει ανάλογα με την αρχιτεκτονική της μηχανής. Αυτό σημαίνει ότι η ίδια λειτουργικότητα είναι αναγκαίο να υλοποιηθεί ξεχωριστά για κάθε διαφορετική μηχανή. Τέλος, τα προγράμματα σε γλώσσα μηχανής είναι απευθείας εκτελέσιμα (executable) από τον υπολογιστή.

Assembly: Προκειμένου να διευκολυνθεί ο άνθρωπος στην επικοινωνία του με τον υπολογιστή δημιουργήθηκε η Assembly. Η Assembly είναι κωδικοποιημένη γλώσσα μηχανής, δηλαδή υποστηρίζει το ίδιο σύνολο εντολών με τη γλώσσα μηχανής. Οι εντολές της όμως δεν εκφράζονται ως ακολουθίες δυαδικών ψηφίων αλλά παρουσιάζουν κάποια ομοιότητα με τις λέξεις της φυσικής γλώσσας. Για παράδειγμα, η ακολουθία 1011000001100001 συνιστά ορθή εντολή σε γλώσσα μηχανής. Η ίδια εντολή σε Assembly εκφράζεται ως mov al, 061h που σημαίνει μετακίνησε τη δεκαεξαδική τιμή 61 (97 στο δεκαδικό σύστημα) στον καταχωρητή με το όνομα "al". Είναι προφανές πως ο άνθρωπος ευκολότερα ανακαλεί εντολές τύπου Assembly παρά εντολές σε γλώσσα μηχανής.

Ο κώδικας σε Assembly δεν εκτελείται κατευθείαν από τη μηχανή. Πρέπει προηγουμένως να μετατραπεί σε γλώσσα μηχανής. Η μετατροπή αυτή γίνεται από κατάλληλο συμβολομεταφραστή που ονομάζεται Assembler.

Γλώσσες υψηλού επιπέδου: Η Assembly διευκολύνει στην αναγνωσιμότητα των προγραμμάτων αλλά διατηρεί το μειονέκτημα πως περιορίζεται από το σύνολο εντολών της μηχανής. Οι εντολές αυτές αντιστοιχούν σε στοιχειώδεις λειτουργίες της μηχανής με αποτέλεσμα και το πιο απλό πρόγραμμα σε Assembly να παρουσιάζει αυξημένο μέγεθος και πολυπλοκότητα και να απαιτεί υψηλό φόρτο εργασίας. Αυτός είναι και ο

λόγος που η Assembly χαρακτηρίζεται ως γλώσσα χαμηλού επιπέδου (low level). Για παράδειγμα, ο κώδικας που ακολουθεί είναι σε Assembly x86-64 και τυπώνει στην οθόνη τον χαρακτήρα '!'.

```
push    $0x21       # '!'
mov     $1, %rax    # sys_write call number
mov     $1, %rdi    # write to stdout (fd=1)
mov     %rsp, %rsi  # use char on stack
mov     $1, %rdx    # write 1 char
syscall
add     $8, %rsp    # restore sp
```

Έτσι προέκυψε η ανάγκη για τις γλώσσες υψηλού επιπέδου (high level programming languages). Ήδη το 1957 κατασκευάστηκε η πρώτη γλώσσα υψηλού επιπέδου, η Fortran της IBM. Οι γλώσσες αυτές είναι ακόμη πιο κοντά στον άνθρωπο σε σχέση με την Assembly και υποστηρίζουν περισσότερες και πιο σύνθετες εντολές. Για παράδειγμα, σε σύγκριση με την Assembly, ο κώδικας Java που εκτυπώνει τον χαρακτήρα '!' είναι System.out.print('!'). Οι γλώσσες υψηλού επιπέδου αυξάνουν δραματικά την παραγωγικότητα κατά την ανάπτυξη εφαρμογών, καθώς ο κώδικας είναι απλούστερος σε σύγκριση με μία γλώσσα μηχανής και Assembly, ενώ δεν απαιτούν να γραφεί διαφορετικό πρόγραμμα για κάθε διαφορετική μηχανή. Ο κώδικας που εκφράζεται σε γλώσσα υψηλού επιπέδου ή Assembly ονομάζεται πηγαίος κώδικας (source code). Σήμερα υπάρχει μεγάλο πλήθος γλωσσών υψηλού επιπέδου με κυριότερους αντιπροσώπους τις C, C++, C#, C objective, Java, Python, Javascript, PHP.

Μεταγλωττιστής: Είναι ευνόητο πως ο πηγαίος κώδικας δεν είναι εκτελέσιμος από τη μηχανή. Ο μεταγλωττιστής (compiler) λαμβάνει ως είσοδο πηγαίο κώδικα και παράγει κώδικα σε γλώσσα μηχανής. Με αυτόν τον τρόπο κωδικοποιείται μια λειτουργία μια φορά σε γλώσσα υψηλού επιπέδου και από αυτήν την κωδικοποίηση μπορεί να παραχθεί εκτελέσιμος κώδικας για κάθε μηχανή που διαθέτει τον αντίστοιχο μεταγλωττιστή. Ένα από τα πλεονεκτήματα του μεταγλωττιστή είναι πως διευκολύνει την προστασία των πνευματικών δικαιωμάτων του κώδικα, καθώς ο κατασκευαστής του λογισμικού δεν διαθέτει στον πελάτη τον πηγαίο αλλά μόνο τον εκτελέσιμο κώδικα. Επιπλέον, διευκολύνει τον προγραμματιστή, καθώς εντοπίζει μια σειρά από συντακτικά κυρίως λάθη κατά τη φάση της μεταγλώττισης (compile time).

Διασυνδέτης: Συχνά, ο πηγαίος κώδικας μίας εφαρμογής αποτελείται από περισσότερα από ένα αρχεία. Κάθε ένα από αυτά τα αρχεία μεταγλωττίζεται χωριστά και για κάθε ένα παράγεται ένα χωριστό αρχείο. Προκειμένου όμως η εφαρμογή να τρέξει, προκύπτει η ανάγκη τα αρχεία που παράγονται από τον μεταγλωττιστή να διασυνδεθούν μεταξύ τους. Την εργασία αυτή αναλαμβάνει ο διασυνδέτης (linker).

Διερμηνευτής: Ο διερμηνευτής (Interpreter) επίσης μεταφράζει τον πηγαίο κώδικα σε εκτελέσιμο αλλά το κάνει κατά τον χρόνο εκτέλεσης (run time), δηλαδή ο διερμηνευτής κατά τον χρόνο εκτέλεσης απαιτεί διαθέσιμο τον πηγαίο κώδικα που μεταφράζει και εκτελεί εντολή προς εντολή. Επομένως ο διερμηνευτής δεν εντοπίζει τα συντακτικά λάθη παρά μόνο κατά τον χρόνο εκτέλεσης του προγράμματος, ενώ δεν παρέχει προστασία των πνευματικών δικαιωμάτων του πηγαίου κώδικα. Επιπλέον, ένα πρόγραμμα σε γλώσσα διερμηνευτή απαιτεί μεγαλύτερο χρόνο εκτέλεσης από το ίδιο πρόγραμμα μεταγλωττισμένο.

Εικονική μηχανή: Όπως αναφέραμε προηγουμένως, ο μεταγλωττιστής μεταφράζει τον πηγαίο κώδικα σε εκτελέσιμο. Πρόκειται για περίπλοκο λογισμικό και η υλοποίησή του απαιτεί αρκετή εργασία και υψηλό κόστος. Όταν μάλιστα θέλουμε τα προγράμματα που αναπτύσσουμε σε μια γλώσσα να είναι εκτελέσιμα σε πολλές πλατφόρμες, είμαστε αναγκασμένοι να υλοποιήσουμε έναν διαφορετικό μεταγλωττιστή για κάθε πλατφόρμα. Μια εναλλακτική λύση είναι η αξιοποίηση εικονικής μηχανής. Στην περίπτωση αυτή έχουμε συνδυασμό ενός μεταγλωττιστή με μια εικονική μηχανή. Ο μεταγλωττιστής όμως δεν παράγει κώδικα εκτελέσιμο απευθείας από τη μηχανή. Αντίθετα, παράγει έναν ενδιάμεσο κώδικα, γνωστό ως bytecode, ο οποίος αποτελεί είσοδο για την εικονική μηχανή που με τη σειρά της παράγει τον εκτελέσιμο κώδικα. Επιπλέον, ο ενδιάμεσος κώδικας δεν διασυνδέεται από έναν τυπικό διασυνδέτη αλλά από τον επονομαζόμενο φορτωτή κλάσης (class loader). Η τεχνολογία αυτή έχει υιοθετηθεί από την Java με την υλοποίηση της Java Virtual Machine ή JVM για συντομία.